

💡 Bootstrap procedure (recap)

- Initialize B (number of bootstrap samples, typically ≥ 1000) and the data $D = \{x_1, \dots, x_n\}$.
- For $b = 1, \dots, B$:
 - Draw D_b^* by sampling **with replacement** from D (same size n).
 - Compute $\hat{\theta}_b$, the statistic of interest, on D_b^* .
- Collect $\hat{\Theta} = [\hat{\theta}_1, \dots, \hat{\theta}_B]$ — this approximates the *sampling distribution* of $\hat{\theta}$.

Today, you will use $\hat{\Theta}$ to build **three different types of bootstrap confidence intervals** for a statistic (and to a classical analytic CI), and then take a first look at how bootstrap ideas power Machine Learning.

Roadmap

Section	What you'll do
1. Warm-up	Simulate data, build a generic bootstrap approximation of a sampling distribution
2. Four CIs	Implement the Percentile, Basic (reflected), Accelerated, and Bias-corrected approximation CIs, and Bias-corrected and accelerated
3. Compare	Put all CIs side-by-side, compare to classical analytic CI for a statistic with no closed form
4. Bootstrap in ML	A first taste of Bagging: does averaging many weak models reduce variance?
5. Going further (<i>take home, ~30-45 min</i>)	Coverage simulation, a bootstrap CI for the variance of B vs. bagging variance

Sections 1-4 are designed for our **75-minute session**; Section 5 is for you to finish **at home**.

Run the cell below to load the packages we need.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.tree import DecisionTreeRegressor
```

1. Warm-up: Simulating Data & the Bootstrap Sampling Distribution [🔗](#)

1.1. A Small, Skewed Sample

Imagine we tracked the **session duration** (in minutes) of $n = 25$ visitors on a website. Session durations are **right-skewed**: most visits are short, a few are very long. Run the cell below to simulate such a sample (the *one and only* observed dataset D , exactly as in the lecture).

```
n = 25
```

```
def bootstrap_estimates(data, stat_func, B=2000):
    """
    Draw B bootstrap samples (with replacement) from `data` and
    return an array of B values of stat_func evaluated on each sample.
    """
    n = len(data)
    estimates = np.zeros(B)
    for b in range(B):
        sample_b = None # TODO: draw one bootstrap sample of size n, with replacement
        estimates[b] = None # TODO: apply stat_func to sample_b
    return estimates
```

C. Use your function to draw $B = 2000$ bootstrap estimates of the **mean** session duration. Plot a bootstrap sampling distribution $\hat{\Theta}$, and report the **bootstrap standard error** $SE(\hat{\theta}) = sd(\hat{\Theta})$.

```
B = 2000
theta_hat = session_times.mean() # the original point estimate, from D itself

theta_boot_mean = None # TODO: bootstrap_estimates(...)
se_boot = None # TODO: standard deviation of theta_boot_mean (use ddof=1)
```

2. Constructing Three Types of Bootstrap Confidence Intervals

From lecture, once we have $\hat{\Theta} = [\hat{\theta}_1, \dots, \hat{\theta}_B]$, there are several ways to turn it into a $(1 - \alpha)$ confidence interval. We'll implement three of them, using the **original** point estimate $\hat{\theta}$ (from D itself) and the bootstrap

💡 Three CI recipes

Let $\hat{\theta}_{(q)}$ denote the q -th percentile of $\hat{\Theta}$, and $SE = sd(\hat{\Theta})$. Let $\alpha \in (0, 1)$ be the error tolerance, therefore confidence level, one has:

- **Percentile CI:** just use the empirical quantiles of $\hat{\Theta}$ directly:

$$CI_{\text{perc}} = [\hat{\theta}_{(\alpha/2)}, \hat{\theta}_{(1-\alpha/2)}]$$

- **Basic (reflection) CI:** reflect the bootstrap quantiles around the original estimate $\hat{\theta}$ — this corrects for bias between $\hat{\theta}$ and the center of $\hat{\Theta}$:

$$CI_{\text{basic}} = [2\hat{\theta} - \hat{\theta}_{(1-\alpha/2)}, 2\hat{\theta} - \hat{\theta}_{(\alpha/2)}]$$

- **Normal-approximation CI:** assume $\hat{\Theta}$ is approximately Normal and just use its standard deviation with $q_{1-\alpha/2} = \Phi^{-1}(1 - \alpha/2)$:

$$CI_{\text{norm}} = [\hat{\theta} - q_{1-\alpha/2} \cdot SE, \hat{\theta} + q_{1-\alpha/2} \cdot SE]$$

All three use the *same* $\hat{\Theta}$ — they just summarize it differently. When $\hat{\Theta}$ is symmetric and unbiased, they all roughly agree.

```

        lower = None # TODO
        upper = None # TODO
        return lower, upper

ci_perc = percentile_ci(theta_boot_mean)
print(f"Percentile 95% CI: [{ci_perc[0]:.3f}, {ci_perc[1]:.3f}]")

```

2.2. Basic (Reflection) CI

B. Complete `basic_ci` below. Notice how it uses `theta_hat` (from the original data) explicitly, not

```

def basic_ci(theta_boot, theta_hat, alpha=0.05):
    lower_perc, upper_perc = percentile_ci(theta_boot, alpha)
    lower = None # TODO
    upper = None # TODO
    return lower, upper

ci_basic = basic_ci(theta_boot_mean, theta_hat)
print(f"Basic 95% CI: [{ci_basic[0]:.3f}, {ci_basic[1]:.3f}]")

```

2.3. Normal-Approximation CI

C. Complete `normal_ci` below. Use `stats.norm.ppf` for the quantile $q_{1-\alpha/2}$.

```

def normal_ci(theta_hat, se, alpha=0.05):
    z = None # TODO: stats.norm.ppf(1 - alpha/2)
    lower = None # TODO
    upper = None # TODO
    return lower, upper

ci_norm = normal_ci(theta_hat, se_boot)
print(f"Normal-approx 95% CI: [{ci_norm[0]:.3f}, {ci_norm[1]:.3f}]")

```

2.4 Bias-corrected CI

D. Complete `bias_corrected_CI` below.

```

def bias_corrected_ci(theta_hat, alpha=0.05):
    bias = None # TODO: compute bias
    c0 = None # TODO: Compute correction c0
    q = None # TODO: stats.norm.ppf(1 - alpha/2)
    aL = None # TODO: compute lower bound of percentage
    aU = None # TODO: compute upper bound of percentage
    lower = None # TODO
    upper = None # TODO
    return lower, upper

ci_bias_corrected = bias_corrected_ci(theta_hat)
print(f"Normal-approx 95% CI: [{ci_bias_corrected[0]:.3f}, {ci_bias_corrected[1]:.3f}]")

```

3. Comparing the Confidence Intervals

horizontal error bars), together with $\hat{\theta}$.

```
ci_dict = {
    'Percentile': ci_perc,
    'Basic': ci_basic,
    'Normal-approx': ci_norm,
    'Bias-correction': ci_bias_corrected,
    'Classical t': ci_t,
}

fig, ax = plt.subplots(figsize=(7, 4))
for i, (name, (lo, hi)) in enumerate(ci_dict.items()):
    center = None # TODO: midpoint of (lo, hi)
    half_width = None # TODO: half of (hi - lo)
    ax.errorbar(center, i, xerr=half_width, fmt='o', capsize=5, color='#345a8b')
ax.axvline(theta_hat, color='gray', linestyle='--', label=r'$\hat{\theta}$')
ax.set_yticks(range(len(ci_dict))); ax.set_yticklabels(ci_dict.keys())
ax.set_xlabel('Mean session duration (minutes)'); ax.set_title('95% CIs for the mean')
ax.legend(); plt.tight_layout(); plt.show()
```

C. In a markdown cell, briefly answer:

- i. Which interval is widest? Narrowest? Does that surprise you given $n = 25$ and the skewness?
- ii. The Basic and Percentile CIs are both built from the same $\hat{\Theta}$ — why can they still give different results?

C. Your answer here:

- i. ...
- ii. ...

3.3. Where Bootstrap Really Shines: A Statistic With No Simple Formula

The t -interval above only works because we have a textbook formula for the standard error of the mean. The bootstrap t -interval doesn't need one — it works for (almost) **any** statistic. Let's try the **median**, for which there is no simple formula.

D. Repeat the bootstrap + three-CI pipeline for `np.median` instead of `np.mean` (reuse your function `bootstrap_cis` for CIs).

```
theta_hat_med = np.median(session_times)
theta_boot_med = None # TODO: compute median instead.
se_boot_med = None # TODO

ci_perc_med = None # TODO
ci_basic_med = None # TODO
ci_norm_med = None # TODO

print(f"Median (theta_hat):      {theta_hat_med:.3f}")
print(f"Percentile 95% CI:      [{ci_perc_med[0]:.3f}, {ci_perc_med[1]:.3f}]")
print(f"Basic 95% CI:              [{ci_basic_med[0]:.3f}, {ci_basic_med[1]:.3f}]")
print(f"Normal-approx 95% CI:      [{ci_norm_med[0]:.3f}, {ci_norm_med[1]:.3f}]")
```

E. In one or two sentences: why is there no simple t -interval equivalent for the median, and why does the bootstrap t -interval work?

```

y_train = np.sin(x_train) + np.random.normal(0, 0.3, n_pts) # For training
x_grid = np.linspace(0, 10, 200).reshape(-1, 1) # For testing

plt.scatter(x_train, y_train, color='#345a8b', label='Training data')
plt.plot(x_grid, np.sin(x_grid), color='black', linestyle='--', label='True function')
plt.legend(); plt.title('A small, noisy regression dataset'); plt.show()

```

A. Train one `DecisionTreeRegressor` (`max_depth=4`) on the full training set, and plot its prediction against the true function. Decision trees trained on few data points tend to be quite unstable/wiggly for the next part.

```

tree_single = DecisionTreeRegressor(max_depth=4, random_state=605)
tree_single.fit(None, None) # TODO: fit on x_train (reshaped to a column) and y_train
y_single = None # TODO: predict on x_grid

plt.scatter(x_train, y_train, color='#345a8b', alpha=0.5)
plt.plot(x_grid, np.sin(x_grid), color='black', linestyle='--', label='True function')
plt.plot(x_grid, y_single, color='#d97f2e', label='Single tree')
plt.legend(); plt.title('A single decision tree'); plt.show()

```

B. Now build a small **Bagging ensemble:** train $B = 100$ trees, each on an independent bootstrap sample (`y_train`) (sample the **indices** with replacement, size n), then average their predictions on `x_grid` and average against the single tree and the truth.

```

B_bag = 100
predictions = np.zeros((B_bag, len(x_grid)))

for b in range(B_bag):
    idx = None # TODO: sample indices with replacement to generate bootstrap sample
    x_b, y_b = x_train[idx], y_train[idx] # select those bootstrap sample from the
    tree_b = None # TODO: build the tree on the bootstrap samples
    tree_b.fit(None, None) # Fit the model on the bootstrap data
    predictions[b] = None # TODO: predict it on x_grid

y_bagged = None # TODO: average predictions over the B_bag trees (axis=0)

plt.scatter(x_train, y_train, color='#345a8b', alpha=0.4)
plt.plot(x_grid, np.sin(x_grid), color='black', linestyle='--', label='True function')
plt.plot(x_grid, y_single, color='#d97f2e', label='Single tree')
plt.plot(x_grid, y_bagged, color='#2e8b57', linewidth=2, label=f'Bagged ({B_bag} trees)')
plt.legend(); plt.title('Single tree vs. Bagging'); plt.show()

```

C. Compute the **pointwise variance** across the $B = 100$ bootstrap-trained trees at each grid point (`predictions.var(axis=0)`), and report its average. Then, in a markdown cell, connect what you know about the variance formula $\mathbb{V}(\bar{f}) = \sigma^2 \left[\rho + \frac{1-\rho}{B} \right]$ — does averaging make the prediction perfectly stable, or not?

```

pointwise_var = None # TODO: compute variance of the prediction using 'predictions'
print(f"Average pointwise variance across the ensemble: {pointwise_var.mean():.4f}")

```

D. Your answer here:

...

- iii. records whether each CI contains the true mean. Report the **empirical coverage** (fraction of the true mean) for each method. (*This will take a little while to run — that's fine, let it finish*)

```
true_mu, true_sigma = 1.6, 0.7
true_mean = None # TODO: apply formula of expectation in question A. above in here.
print(f"True population mean: {true_mean:.4f}")

n_repeats = 200
B_small = 500
covered = {'Percentile': 0, 'Basic': 0, 'Normal-approx': 0}

for r in range(n_repeats):
    sample_r = None # TODO: draw a fresh lognormal sample, size n, params (true_mu, true_sigma)
    theta_hat_r = sample_r.mean()
    theta_boot_r = bootstrap_estimates(sample_r, np.mean, B_small)
    se_r = theta_boot_r.std(ddof=1)

    lo_p, hi_p = percentile_ci(theta_boot_r)
    lo_b, hi_b = basic_ci(theta_boot_r, theta_hat_r)
    lo_n, hi_n = normal_ci(theta_hat_r, se_r)

    covered['Percentile'] += (lo_p <= true_mean <= hi_p)
    covered['Basic'] += (lo_b <= true_mean <= hi_b)
    covered['Normal-approx'] += (lo_n <= true_mean <= hi_n)

for method, count in covered.items():
    print(f"{method:15s}: empirical coverage = {count/n_repeats:.3f}")
```

C. Is the empirical coverage close to the nominal 95%? If one method's coverage is noticeably off, can you connect that back to what you observed about skewness in Section 1?

C. *Your answer here:*

...

5.2. Bootstrap CI for a Correlation Coefficient

Bootstrap is especially handy for statistics like the **Pearson correlation** r , which has an awkward (bounded in $[-1, 1]$, skewed when $|r|$ is large) — an exact analytic CI exists (Fisher's z -transform) but is wrong, whereas the bootstrap recipe is *exactly the same as before*.

A. Run the cell below to simulate $n = 40$ paired observations (u_i, v_i) with true correlation around

```
np.random.seed(47)
n_corr = 40
mean_uv = [0, 0]
cov_uv = [[1, 0.6], [0.6, 1]]
uv = np.random.multivariate_normal(mean_uv, cov_uv, size=n_corr)
u, v = uv[:, 0], uv[:, 1]

plt.scatter(u, v, color='#345a8b'); plt.xlabel('u'); plt.ylabel('v')
plt.title(f'Sample correlation r = {np.corrcoef(u, v)[0,1]:.3f}'); plt.show()
```

5.3. Bonus: How Does B Affect Bagging's Variance Floor?

Recall from lecture: with pairwise model correlation ρ , $\mathbb{V}(\bar{f}) = \sigma^2 \left[\rho + \frac{1-\rho}{B} \right] \rightarrow \rho\sigma^2$ as $B \rightarrow \infty$ but only up to a point.

A. Using the same setup as Section 4, repeat the bagging experiment for a few different ensemble sizes $B = [1, 5, 20, 50, 100, 300]$. For each B , train the ensemble (you can reuse your loop from Section 4) and compute the **average pointwise variance** across grid points. Plot average variance vs. B . Does it plateau, or does it keep decreasing? Does it predict?

```
B_values = [1, 5, 20, 50, 100, 300, 500]
avg_variances = []

for B_val in B_values:
    preds = np.zeros((B_val, len(x_grid)))
    for b in range(B_val):
        idx = None # generate random indices for bootstrap sample
        x_b, y_b = None, None # draw bootstrap sample
        tree_b = DecisionTreeRegressor(max_depth=4, random_state=b)
        tree_b.fit(None, None) # fit model on the bootstrap sample
        preds[b] = None # Prediction
    avg_variances.append(None) # TODO: average pointwise variance for this B_val

plt.plot(B_values, avg_variances, marker='o', color='#345a8b')
plt.xlabel('Number of bootstrap trees B'); plt.ylabel('Average pointwise variance')
plt.title('Does more trees keep reducing variance forever?'); plt.show()
```

Wrap-up

In this lab you:

- Built a **generic bootstrap resampler** and used it to approximate the sampling distribution of a statistic.
- Implemented and compared **four types of bootstrap CIs** — Percentile, Basic (reflection), and accelerated — against a classical t -interval (Sections 2–3).
- Saw bootstrap's real advantage: it works just as easily for statistics with **no simple formula**, like the median (Section 3.3).
- Connected bootstrap resampling to **Bagging** in Machine Learning, and saw how averaging bootstrap models reduces prediction variance (Section 4).
- (At home) Verified **empirical coverage** of your CIs, built a bootstrap CI for a **correlation coefficient**, and saw how the bagging **variance floor** depends on B (Section 5).

The core idea carries through every section: **treat your one dataset as a stand-in for the population, sample it many times, and let the resulting variation tell you how much to trust your estimate.**

